

Practice Interview

Objective

The partner assignment aims to provide participants with the opportunity to practice coding in an interview context. You will analyze your partner's Assignment 1. Moreover, code reviews are common practice in a software development team. This assignment should give you a taste of the code review process.

Group Size

Each group should have 2 people. You will be assigned a partner

Part 1:

You and your partner must share each other's Assignment 1 submission.

Part 2:

Create a Jupyter Notebook, create 6 of the following headings, and complete the following for your partner's assignment 1:

- Paraphrase the problem in your own words.

```
In [ ]: # Your answer here
```

The question is to check if there is duplicate numbers in the binary tree. If there is, return the duplicated number. If there are multiple duplicated numbers, return the one closest to the root. IF there is no duplicates, return -1.

- Create 1 new example that demonstrates you understand the problem. Trace/walkthrough 1 example that your partner made and explain it.

```
In [ ]: # Your answer here
```

input: root = [1,2,3,4,5,5,6] output 5

- Copy the solution your partner wrote.

```
In [ ]: # Your answer here
from collections import deque
# Definition for a binary tree node.
class TreeNode(object):
    def __init__(self, val = 0, left = None, right = None):
        self.val = val
        self.left = left
```

```

        self.right = right

def is_duplicate(root: TreeNode) -> int:
    if not root:
        return -1

    # BFS traversal
    queue = deque([(root, 0)]) # Queue stores (node, depth)
    seen = {} # Dictionary to track occurrences of values

    while queue:
        node, depth = queue.popleft()

        # Check if the value has already been seen
        if node.val in seen:
            return node.val

        # Record the value and its depth
        seen[node.val] = depth

        # Add children to the queue
        if node.left:
            queue.append((node.left, depth + 1))
        if node.right:
            queue.append((node.right, depth + 1))

    # No duplicates
    return -1

```

```

In [ ]: root = TreeNode(1,
                        TreeNode(2,TreeNode(4), TreeNode(5)),
                        TreeNode(3, TreeNode(5),TreeNode(6)))

```

```

In [ ]: print(is_duplicate(root))

```

5

- Explain why their solution works in your own words.

```

In [ ]: # Your answer here

```

BFS processes nodes level by level, starting from the root. This ensures the duplicate closest to the root is found first. The seen dictionary efficiently tracks all previously encountered values. Checking and inserting into the dictionary both take $O(1)$ on average. As soon as a duplicate is found, the function exits with the duplicate value. This ensures minimal traversal. The queue ensures all nodes are visited in the correct order.

- Explain the problem's time and space complexity in your own words.

```

In [ ]: # Your answer here

```

Time Complexity: BFS processes each node exactly once and these operation takes $O(1)$ each. The function processes each node once and each operation is $O(1)$. the total time complexity is $O(n)$.

Space Complexity: Queue:

The maximum size of the queue is determined by the number of nodes at the widest level of the binary tree. For a balanced binary tree, this is approximately half the number of nodes at the last level $O(n/2)$, which simplifies to: $O(n)$. For a skewed binary tree, the queue can grow to $O(n)$. The seen dictionary stores unique node values. In the worst case, all n nodes have unique values, so the dictionary size will also be $O(n)$. Total Space Complexity: The queue and dictionary dominate the space usage, so the total space complexity is: $O(n)$

- Critique your partner's solution, including explanation, and if there is anything that should be adjusted.

```
In [ ]: # Your answer here
#I think her answer is great. the use of BFS align the problem.
#The function correctly handles the case where the tree is empty (if not root: return -1
```

Part 3:

Please write a 200 word reflection documenting your process from assignment 1, and your presentation and review experience with your partner at the bottom of the Jupyter Notebook under a new heading "Reflection." Again, export this Notebook as pdf.

Reflection

My assignment in assignment-1 is the question 3: Missing Number in Range, the goal was to identify which numbers were missing from the range $[0, n]$ of a given list. If there is no missing number, the result should return -1. The process began with paraphrasing the problem to ensure clarity, creating new test examples to validate understanding, and then coding the solution. This problem involved both theoretical understanding and practical implementation. The solution uses a combination of Python's built-in functions and data structures for efficiency. I implemented the solution using Python's set data structure for fast lookups and a list comprehension to identify missing numbers efficiently. The use of `max(nums)` ensured that the range of numbers was correctly defined.

My partner's problem involved detecting the first duplicate value in a binary tree, focusing on finding the one closest to the root. Her approach used Breadth-First Search (BFS) for traversal and a dictionary to track seen values. The use of BFS is really smart move. This naturally finds the closest duplicate. Critiquing my partner's solution deepened my understanding of BFS and the trade-offs between dictionaries and sets for duplicate detection. It also highlighted the importance of clear communication, both in code comments and during peer discussions.

Evaluation Criteria

We are looking for the similar points as Assignment 1

- Problem is accurately stated
- New example is correct and easily understandable
- Correctness, time, and space complexity of the coding solution

- Clarity in explaining why the solution works, its time and space complexity
- Quality of critique of your partner's assignment, if necessary

Submission Information

🔴 Please review our [Assignment Submission Guide](#) 🔴 for detailed instructions on how to format, branch, and submit your work. Following these guidelines is crucial for your submissions to be evaluated correctly.

Submission Parameters:

- Submission Due Date: `HH:MM AM/PM – DD/MM/YYYY`
- The branch name for your repo should be: `assignment-2`
- What to submit for this assignment:
 - This Jupyter Notebook (`assignment_2.ipynb`) should be populated and should be the only change in your pull request.
- What the pull request link should look like for this assignment:
`https://github.com/<your_github_username>/algorithms_and_data_structures/pull/`
 - Open a private window in your browser. Copy and paste the link to your pull request into the address bar. Make sure you can see your pull request properly. This helps the technical facilitator and learning support staff review your submission easily.

Checklist:

- ☐ Created a branch with the correct naming convention.
- ☐ Ensured that the repository is public.
- ☐ Reviewed the PR description guidelines and adhered to them.
- ☐ Verify that the link is accessible in a private browser window.

If you encounter any difficulties or have questions, please don't hesitate to reach out to our team via our Slack at `#cohort-3-help`. Our Technical Facilitators and Learning Support staff are here to help you navigate any challenges.
